

Capítulo 20. Abstracciones de Datos: JDBC Funcional (Doobie)

1. Explicación Teórica: Desacoplamiento de Efectos y Seguridad

Doobie es la solución idiomática en el ecosistema de Scala para la interacción con bases de datos relacionales a través de JDBC (Java Database Connectivity). Se describe como una **capa JDBC puramente funcional para Scala**.

El Problema de la Impureza en JDBC

En la programación tradicional, las operaciones de base de datos (E/S o I/O) son inherentemente **impuras** e **imperativas**. Un código que abre una conexión, ejecuta una consulta, maneja el resultado y luego cierra la conexión (a menudo en un bloque `try/catch/finally` en Java,) presenta varios problemas:

1. **Mutabilidad de Estado:** La conexión es un recurso mutable gestionado manualmente.
2. **Efectos Secundarios:** Las excepciones (como `SQLException`) interrumpen el flujo de ejecución del programa, rompiendo el principio de **transparencia referencial** de la Programación Funcional (PF).
3. **Gestión de Recursos (Resource Safety):** El desarrollador debe garantizar manualmente que la conexión se cierre, lo que es propenso a errores.

Doobie: El Efecto como Valor Inmutable

Doobie resuelve estos problemas encapsulando la interacción con la base de datos dentro de la **mónada de efectos** `ConnectionIO[A]`. Al igual que la mónada `IO` (de Cats Effect o ZIO) un valor `ConnectionIO` no ejecuta el código inmediatamente; es una **descripción inmutable** o un **"plano"** (*blueprint*) de la acción que debe realizarse en la base de datos.

La ejecución de este "plano" se delega a una capa externa, el **Transactor** (`Transactor[F]`), que es responsable de:

- Obtener conexiones de un *pool*.
- Manejar las transacciones (commit/rollback).
- Garantizar el **cierre seguro de recursos**, eliminando la necesidad de bloques `finally` explícitos,,.

El enfoque funcional puro de Doobie utiliza el sistema de tipos avanzado de Scala para **comprobar la seguridad del tipo de las consultas SQL en tiempo de compilación**, previniendo errores comunes de mapeo entre Scala y la base de datos.

2. Sintaxis y Tipos Centrales

La sintaxis de Doobie se basa en *for-comprehensions* y *string interpolators* para expresar consultas SQL de forma segura y componible.

A. Tipos Clave

Tipo	Descripción	Función Principal
ConnectionIO[A]	La mónada de efecto principal. Representa una descripción de una acción que se ejecuta en el contexto de una conexión (transacción). A es el valor de retorno (ej. Int , Usuario).	Se utiliza para componer la lógica de negocio que interactúa con la DB.
Transactor[F]	El motor de ejecución. Convierte el valor ConnectionIO[A] en el efecto externo F[A] (donde F es típicamente IO de Cats Effect o ZIO), delegando la gestión de la conexión.	Se gestiona como una dependencia (Environment).

B. Query DSL (String Interpolators)

Doobie utiliza *string interpolators* especiales para inyectar variables de Scala de forma segura y construir consultas, evitando la inyección SQL (SQL Injection).

Interpolador	Sintaxis Gramatical	Propósito
sql (Raw)	sql"SELECT * FROM tabla"	Se usa para el texto SQL estático.
fr (Fragmento)	fr"SELECT * FROM tabla WHERE id = \${miId}"	El interpolador preferido. Permite la inyección segura de valores (los convierte en ? en el SQL, y los valores se pasan como parámetros de JDBC).

3. Ejemplos de Código Realistas: CRUD Funcional

Este ejemplo muestra la composición de operaciones de base de datos (creación de esquema, inserción, y consulta) utilizando la mónada ConnectionIO .

```
1 import cats.effect.IO // Efecto externo (monad IO)
2 import doobie._
3 import doobie.implicit._ // Habilita los interpoladores 'sql' y 'fr'
4 import doobie.util.transactor.Transactor
```

```

5  import doobie.hikari.HikariTransactor // Transactor de conexión
6
7  // 1. Modelo de Datos (representación de la fila de la DB)
8  case class Usuario(id: Long, nombre: String, email: String)
9
10 // 2. Definición del Transactor (Configuración - Se haría una vez al
    inicio del sistema)
11 // El Transactor es la dependencia que sabe cómo conectarse a la DB.
12 val xa: Transactor[IO] = Transactor.fromDriverManager[IO](
13     "org.postgresql.Driver",
14     "jdbc:postgresql:mydb",
15     "user",
16     "password"
17 )
18
19 // --- OPERACIONES EN CONNECTIONIO ---
20
21 // 3. Crear el Esquema (Efecto de IO)
22 // Un ConnectionIO[Int] que representa el número de filas modificadas (1
    si se crea).
23 val crearTabla: ConnectionIO[Int] =
24     sql"""
25         CREATE TABLE IF NOT EXISTS usuarios (
26             id BIGSERIAL PRIMARY KEY,
27             nombre VARCHAR NOT NULL,
28             email VARCHAR NOT NULL
29         )
30     """.update.run
31
32 // 4. Inserción Segura de Datos (Inyección de Parámetros)
33 // Usamos el interpolador 'fr' (fragmento) para inyectar variables de
    Scala.
34 def insertarUsuario(nombre: String, email: String): ConnectionIO[Usuario]
    =
35     val insercion = fr"INSERT INTO usuarios (nombre, email) VALUES ($nombre,
    $email)"
36
37     insercion.update.withUniqueGeneratedKeys[Long]("id").map { idGenerado =>
38         Usuario(idGenerado, nombre, email) // Devuelve el objeto Usuario con
    el ID real.
39     }
40
41 // 5. Consulta de Datos (Mapeo de Filas)
42 // .query[Usuario] indica a Doobie cómo mapear las columnas al case class
    Usuario.
43 val obtenerUsuarios: ConnectionIO[List[Usuario]] =

```

```

44     sql"SELECT id, nombre, email FROM usuarios".query[Usuario].to[List]
45
46 // --- COMPOSICIÓN Y EJECUCIÓN ---
47
48 // 6. Composición Funcional (for-comprehension)
49 // Componemos las acciones de DB como un único
    ConnectionIO[List[Usuario]].
50 val flujoCompleto: ConnectionIO[List[Usuario]] =
51     for {
52         _ <- crearTabla                                // Se ejecuta (produce
un Int, pero lo ignoramos)
53         _ <- insertarUsuario("Ana García", "ana@test.com")
54         _ <- insertarUsuario("Luis Pérez", "luis@test.com")
55         usuarios <- obtenerUsuarios                    // Obtenemos la lista
56     } yield usuarios
57
58 // 7. Ejecución (al "final del universo")
59 // Transactor.transact lo envuelve en una transacción segura y lo ejecuta.
60 val resultadoIO: IO[List[Usuario]] = flujoCompleto.transact(xa)
61
62 // Para correr la aplicación, se llamaría a 'resultadoIO.unsafeRunSync()'
    o se integraría

```

4. Comparativa con Java o Python

La diferencia fundamental entre Doobie y las herramientas tradicionales de acceso a bases de datos es el **cambio de paradigma de imperativo a puramente funcional**, lo que aumenta la seguridad en tiempo de compilación y la resiliencia en *runtime*.

Aspecto	Java (JDBC o JPA/Hibernate)	Python (SQLAlchemy ORM/Raw)	Scala (Doobie)
Manejo de I/O	Imperativo. Requiere <code>try-catch</code> para excepciones y <code>finally</code> para cerrar <code>Connection</code> y <code>ResultSet</code> .	Orientado a objetos o procedural.	Funcional Puro. La interacción con la DB se modela como un valor <code>ConnectionIO</code> .
Transacciones	Gestión manual de <code>.setAutoCommit(false)</code> y <code>.commit()</code> / <code>.rollback()</code> .	Gestión manual o por capa de abstracción de ORM.	Automática y Declarativa. El <code>.transact(xa)</code> envuelve todo el bloque <code>ConnectionIO</code> en una única transacción atómica.

Aspecto	Java (JDBC o JPA/Hibernate)	Python (SQLAlchemy ORM/Raw)	Scala (Doobie)
Seguridad de SQL	Inyección SQL es un riesgo si se construye el <i>string</i> de consulta manualmente.	Similar a Java.	Seguridad en Tipos. El interpolador <code>fr</code> garantiza que los parámetros de Scala sean <i>bind</i> de JDBC, previniendo inyección SQL.
Composición	Difícil de componer pasos transaccionales sin anidación de bloques o <i>callbacks</i> .	Limitada.	Composición Monádica. Los pasos se encadenan de forma secuencial y legible con <i>for-comprehensions</i> (<code>map / flatMap</code>).
Recursos	Recolección de basura de objetos de conexión lentos (JVM).	N/A.	Gestión de Recursos Segura. Utiliza abstracciones funcionales (<i>Resource</i>) del motor de efectos (<code>IO</code>) para garantizar que la conexión se cierre automáticamente.

5. Best Practices (*The Scala Way*)

El uso de Doobie en Scala 3 representa el camino funcional ideal para manejar la persistencia:

- Mantenimiento de la Pureza:** La lógica de negocio que interactúa con la base de datos debe residir exclusivamente en el tipo `ConnectionIO[A]`. Esto garantiza que las operaciones de la base de datos sean puras, inmutables y no se ejecuten hasta que se llamen a `transact`.
- Uso de `case class` para Mapeo:** Define siempre las filas de la base de datos con `case class`. Doobie puede inferir automáticamente cómo mapear las columnas al `case class` con seguridad de tipos (siempre y cuando los nombres de las columnas coincidan).
- Prevención de Inyección SQL:** Nunca utilices interpolación de *strings* estándar (ej. `s"..."`) para construir consultas SQL que incluyan variables de usuario. Utiliza siempre el interpolador `fr` de Doobie para inyectar valores de forma segura como parámetros de JDBC.

4. **Inyección de Dependencias (Transactor):** El `Transactor[F]` es un recurso de infraestructura que debe tratarse como una **dependencia**. En arquitecturas funcionales modernas (como ZIO o Cats Effect), el Transactor se gestiona como un recurso global disponible a través del sistema de inyección de dependencias (`ZLayer` en ZIO o `Resource[IO, Transactor]` en Cats Effect).

Doobie actúa como un **notario funcional** entre tu código Scala y la base de datos. En lugar de ejecutar órdenes directamente (imperativo), tú le entregas a Doobie una **descripción** detallada de lo que debe hacer (el `ConnectionIO`). El compilador, mediante la seguridad de tipos, certifica que esa descripción es correcta y segura (libre de inyección SQL), y el Transactor se encarga de ejecutarla en un **entorno controlado y a prueba de fallos** (la transacción).